

Representation Learning for Text Sequences

Edwin Simpson

bristol.ac.uk



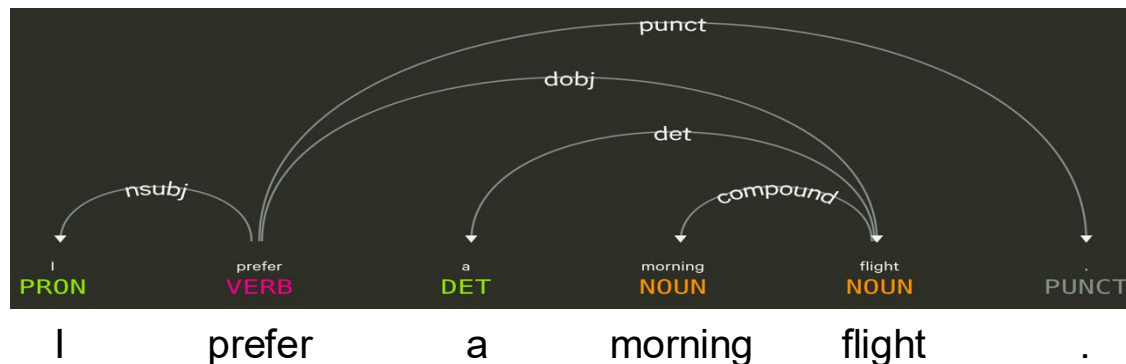
Is Language Important?

- Language can encode
 - Knowledge
 - Events
 - Instructions
 - Computer programs
 - Thought processes
 - Emotions
- Communication between humans and AI systems
- “Language is a way to express thoughts, but thoughts, memories, and reasoning exist without language.” Yann LeCun, March 2024.



What Makes Text ~~Special~~ Challenging?

1. Sequences of **discrete** units (letters, words) form meaning.
2. **Rare** observations: many possible sentences are never observed.
3. Text is **compositional**: words are structured into phrases, which combine to form sentences, and so on.



4. Ambiguity, errors and variations in the way people use language.

Tokenisation

- Aim: split a string of characters into a sequence of discrete **tokens** that are **efficient building blocks for meaning**
- Tokens $\approx$ Words, but what about...
 - Punctuation
 - Abbreviations and acronyms
 - Hashtags and URLs
 - Contractions like ‘don’t’
 - Disfluencies like ‘errr...’
 - Suffixes and prefixes like “re-” or “-isation”
 - Multiword phrases like “New York”
 - Languages without spaces between words, e.g., Thai (ภาษาไทย), Japanese, Chinese
- **Solution:** build a vocabulary of the most common N character sequences (words and sub-words) in a dataset
- At test time, divide the character sequence into the longest matching (sub)words

Meaningful Representations for Processing

- Simple machine learning methods can't recognise meaning from raw token streams
- Can we convert text data to different *representations* to make processing by computer possible?
- Desiderata for a good representation:
 - a. Preserves **only** the relevant aspects of meaning for a particular task, e.g., retrieving documents on topic X versus answering a question
 - b. Generalises to new examples
 - c. Can be computed in a reasonable amount of time
 - d. Works well with downstream models
 - e. **Numerical, continuous**

Can we *learn* numerical representations of language meaning?

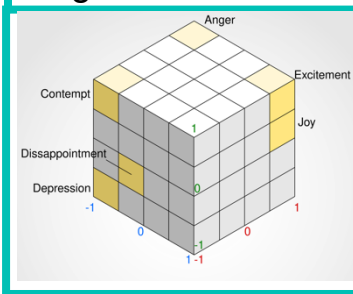


A Brief History

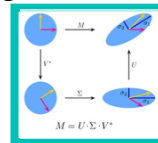
1950s: Distributional Hypothesis,
Joos, Harris, Frith

*“you shall know a word by
the company it keeps” –
Frith 1957*

1957: Valence, arousal, dominance,
Osgood – semantics as a vector



1990: Latent semantic indexing,
Derwester – embeddings based
on co-occurrence



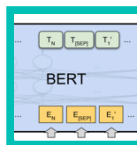
2022-: Chain
of thought,
Wei et al. –
‘thinking’ LMs



2022: InstructGPT,
Ouyang et al. –
Instruction following,
human alignment

2020: GPT3, Brown
et al. – coherence,
in-context learning

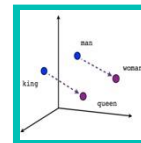
2018: ELMo, Peters et al.
– contextual embeddings



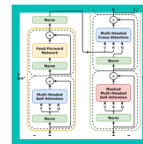
2018: BERT,
Devlin et al. –
pretrain/tune
transformers



2018: ULMFiT,
Howard &
Ruder –
language
model transfer



2013:
word2vec,
Mikolov et al.
– learn
dense word
embeddings
from big data



2017: Transformer,
Vaswani et al. –
scale, attention only

Word2vec: How To Learn From Lots of Data

- Leverage lots of unlabelled data, such as books and Wikipedia
- Design a prediction task that requires encoding meaning, but doesn't need human labels
- Try to **predict a word given its context**
 - Q. What kind of learning is this (supervised, unsupervised...)?
 - **A. Self-supervised**

... lemon, a [tablespoon of apricot jam, a] pinch ...
 c1 c2 t c3 c4

Word2vec: How Do We Learn?

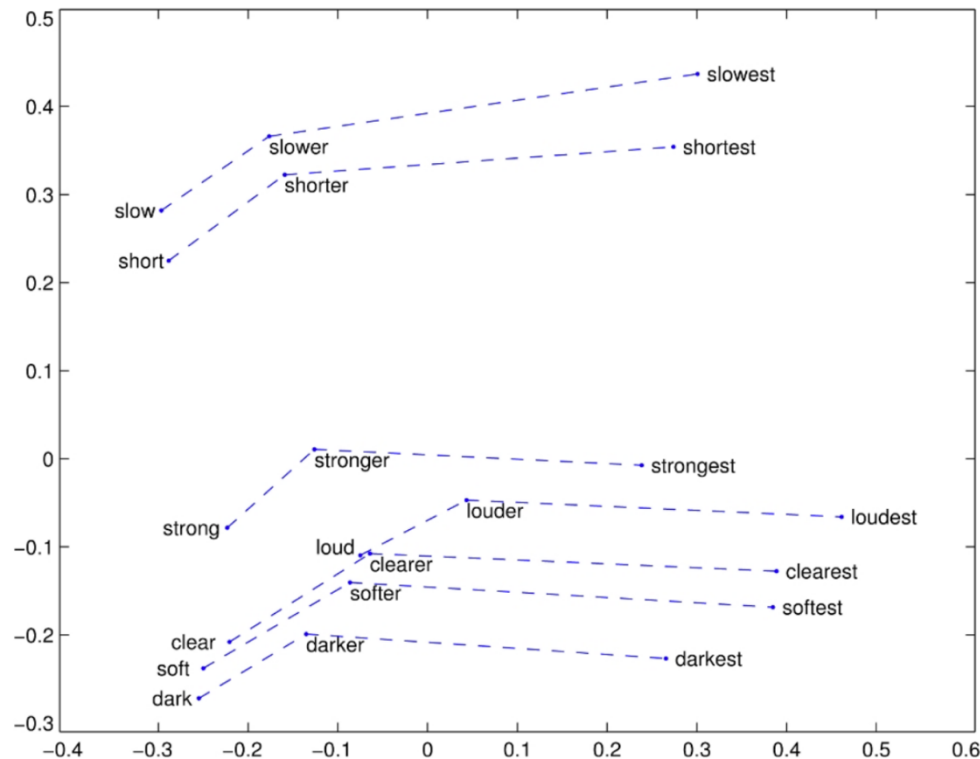
- For every word t in the vocabulary...
- Train a **logistic regression classifier** to predict if a word c_i occurs in the context of t :

$$P(+ \mid \mathbf{c}, \mathbf{t}) = \prod_{i=1}^k \frac{1}{1 + e^{-\mathbf{t} \cdot \mathbf{c}_i}}$$

- Input $\mathbf{c}_i$ and weights $\mathbf{t}$ are vectors that we learn by gradient descent
 - Initialise $\mathbf{c}_i$ and $\mathbf{t}$ randomly
 - Dot product $\mathbf{t} \cdot \mathbf{c}_i$ compares the context and target words
 - Make positive pairs ($\mathbf{c}_i$ and $\mathbf{t}$ from ‘real’ contexts) more aligned
 - Make negative pairs (randomly chosen $\mathbf{c}_i$) less aligned
- Context word vectors $\mathbf{c}_i$ shared across all target word classifiers

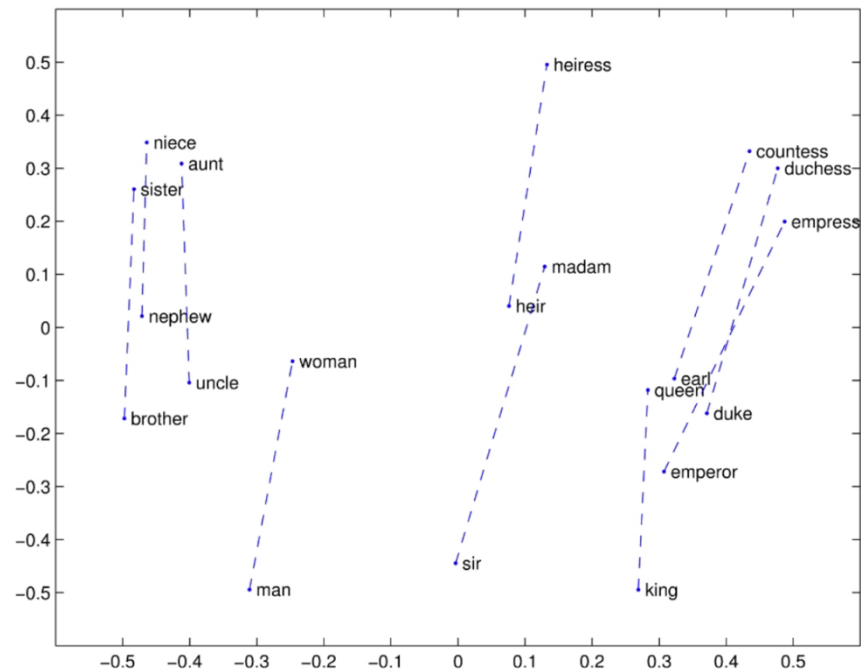
Word2vec: What Does It Produce?

- Output t as an **embedding vector**
- What properties do the embeddings have?
- Embeddings are similar for words with similar contexts
- Semantic relations are represented by subspaces:



Word Embeddings: Word2vec

- Why was word2vec so influential?
 - Properly encode semantic relations (e.g., synonyms, antonyms, etc...)
 - Learn from lots of unlabelled data
 - Useful: can be reused as input layer to a neural network
- What limitations does word2vec have?
 - Static embeddings
 - Polysemous or ambiguous words
 - How to build sentences embeddings?



Can We *Contextualise* Word Embeddings or *Compose* Sentence Embeddings?



Representations in Neural Networks

- Hidden layer produces a vector representation of the inputs
- Backpropagation updates the weights, tuning the representations to separate sentences, e.g., by sentiment

[Jurafsky and Martin, Speech and Language Processing 3](#)

[bristol.ac.uk](#)

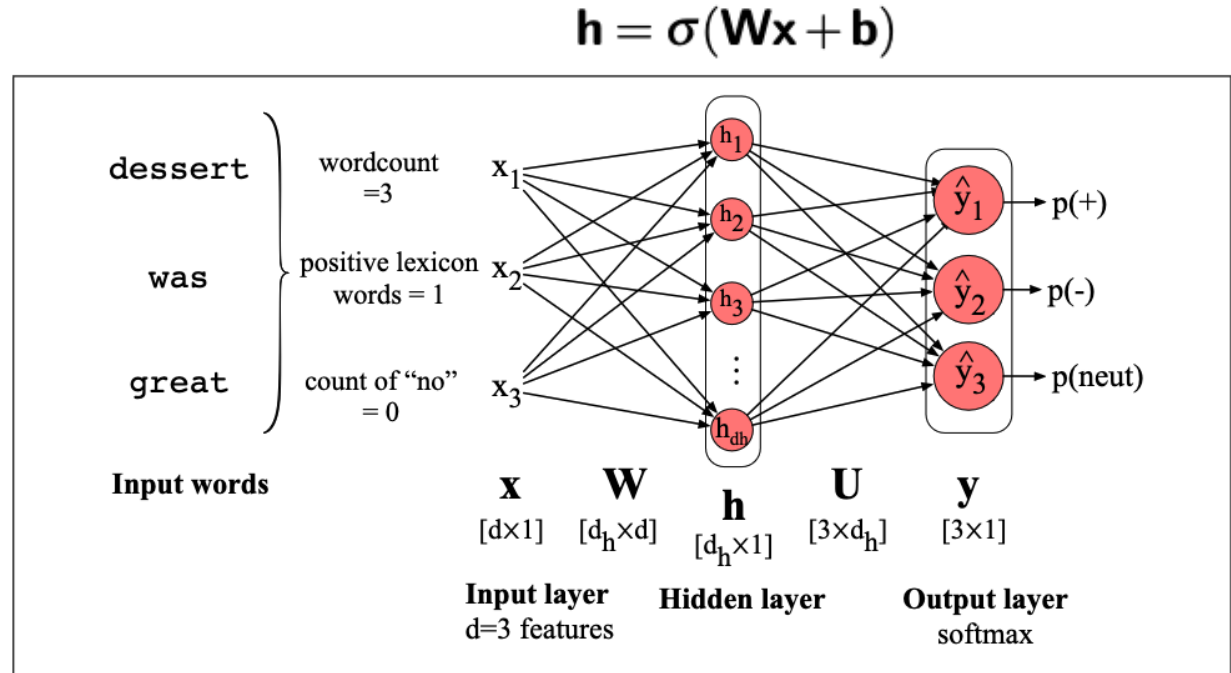
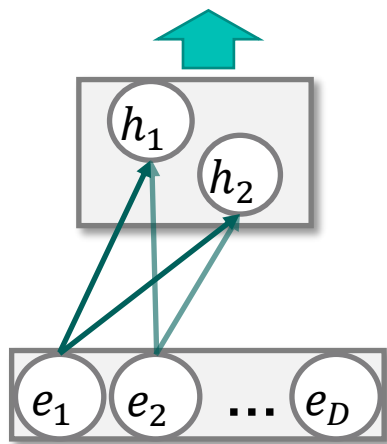


Figure 6.10 Feedforward network sentiment analysis using traditional hand-built features of the input text.

Process Words Sequentially with a Recurrent Neural Network (RNN)

Word embedding



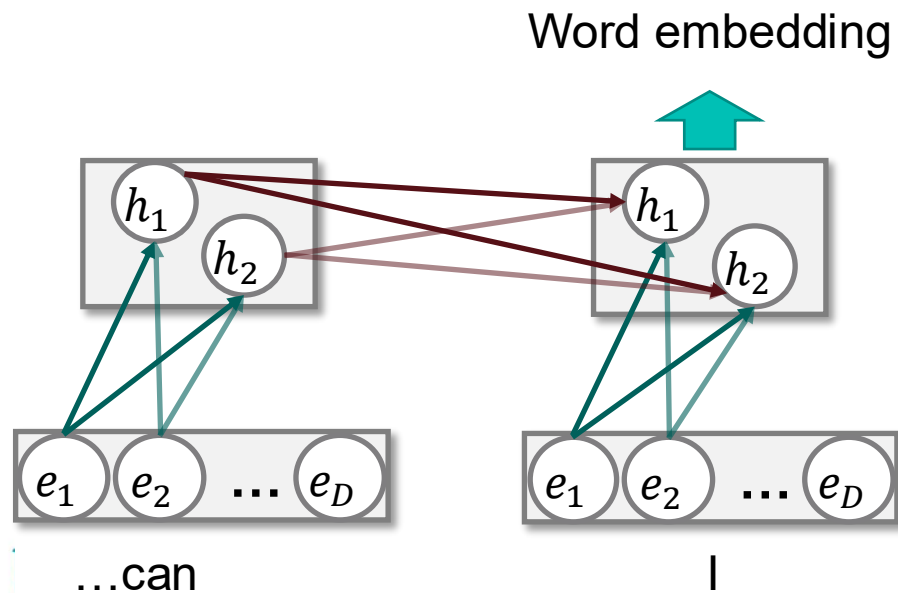
Recurrent layer:
- same weights are
applied to each token
- position independent

...can

|

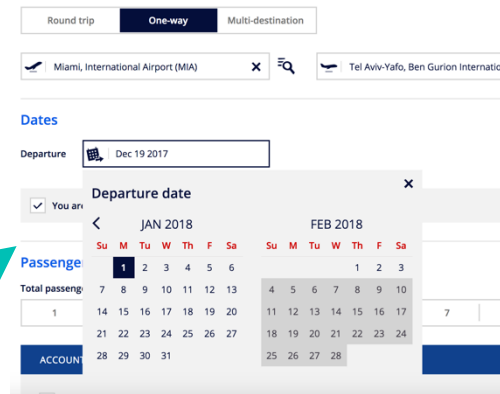
book ...

Process Words Sequentially with a Recurrent Neural Network (RNN)



Recurrent layer:
- same weights are
applied to each token
- position independent

Process Words Sequentially with a Recurrent Neural Network (RNN)



Round trip **One-way** Multi-destination

Miami, International Airport (MIA) Tel Aviv-Yafo, Ben Gurion International

Dates

Departure Dec 19 2017

Departure date

< JAN 2018 FEB 2018

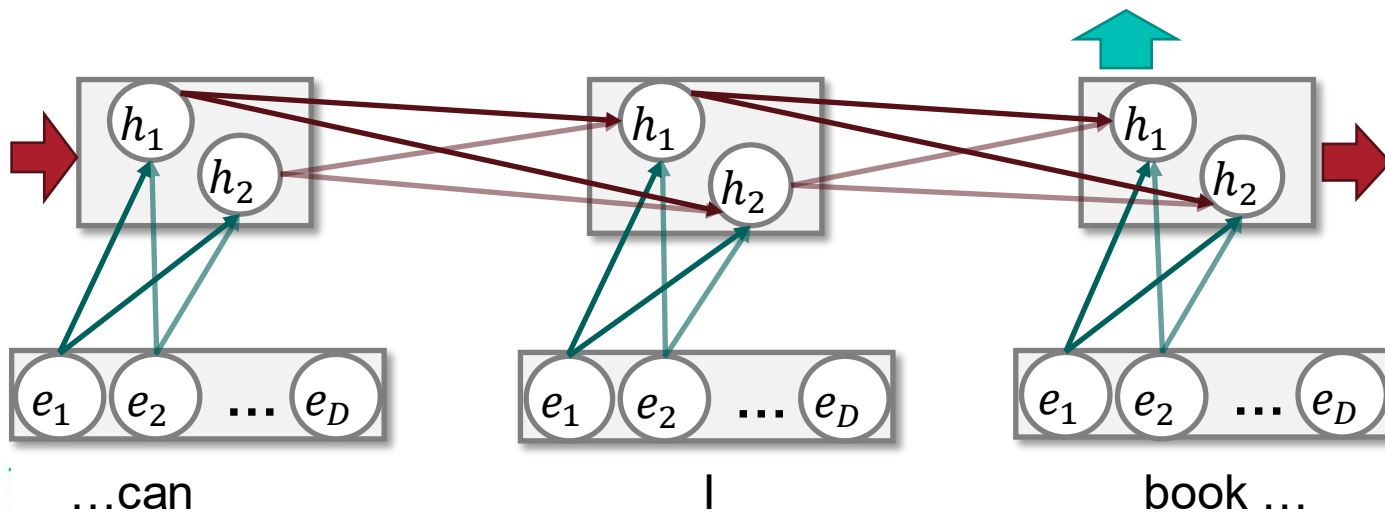
Su	M	Tu	W	Th	F	Sa
1	2	3	4	5	6	
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30	31			

Passenger

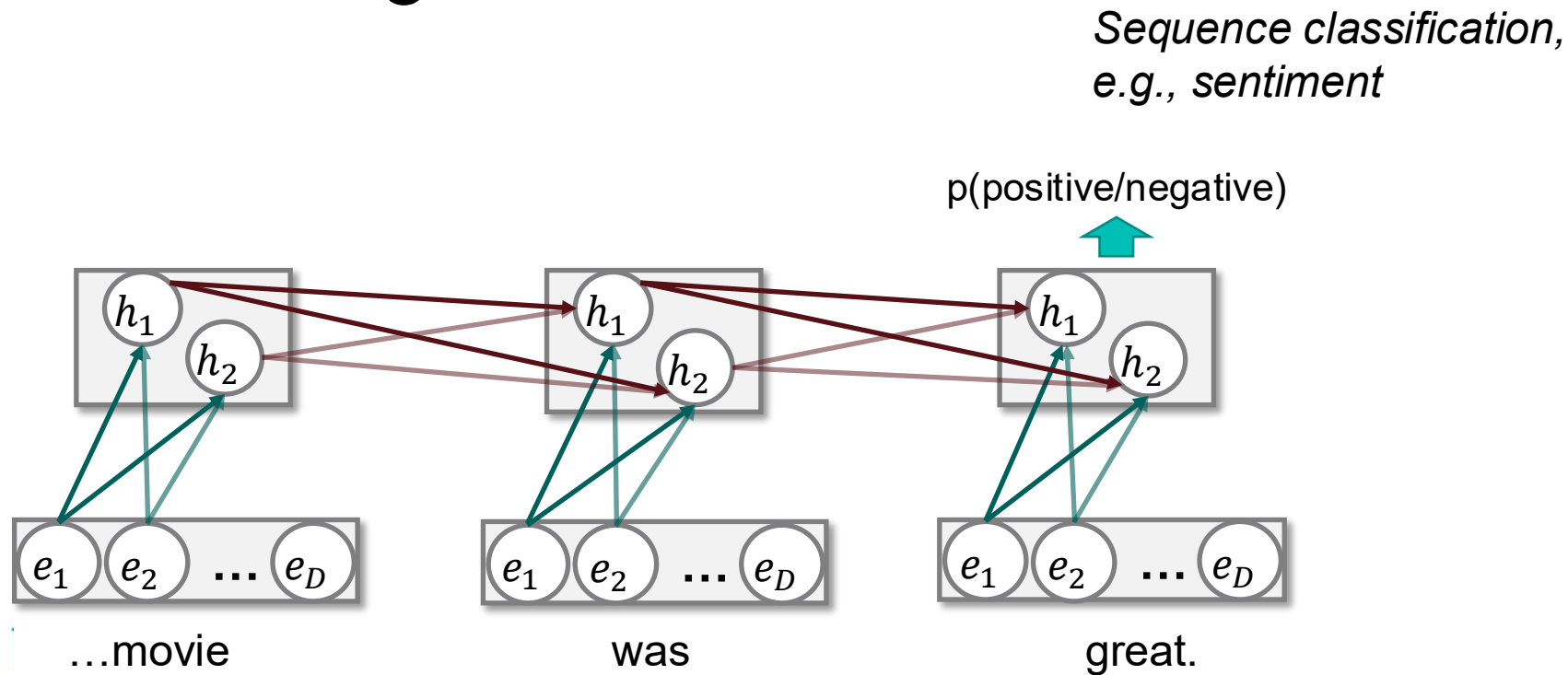
Total passenger 1

ACCOUNT

Contextualised word embedding

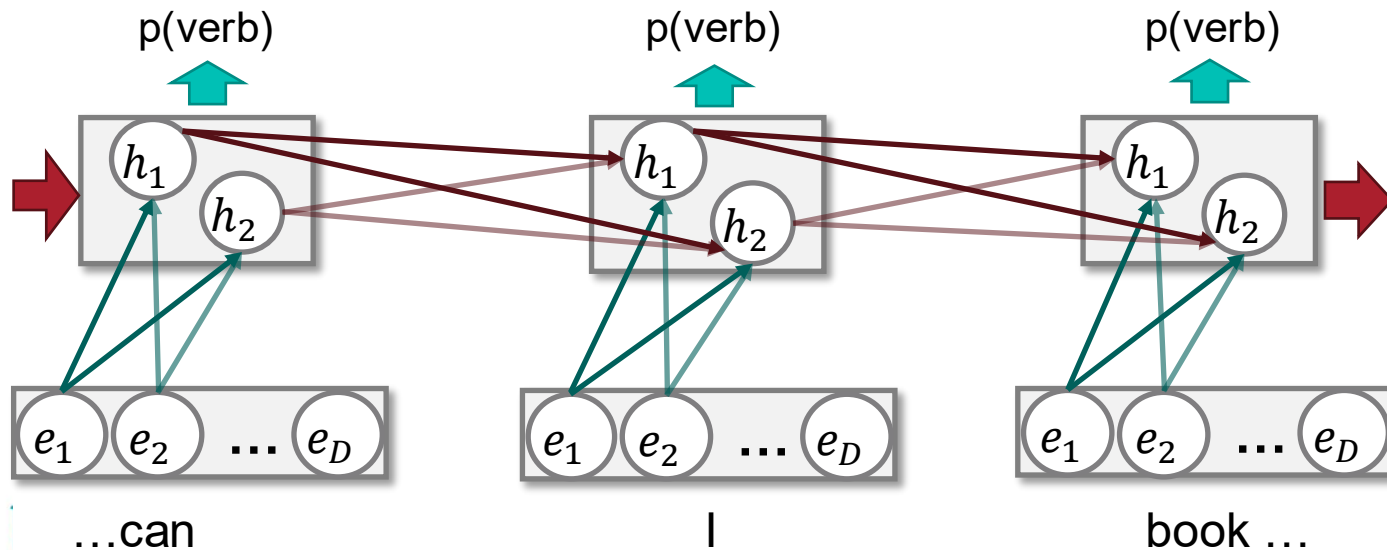


What can you do with these embeddings?

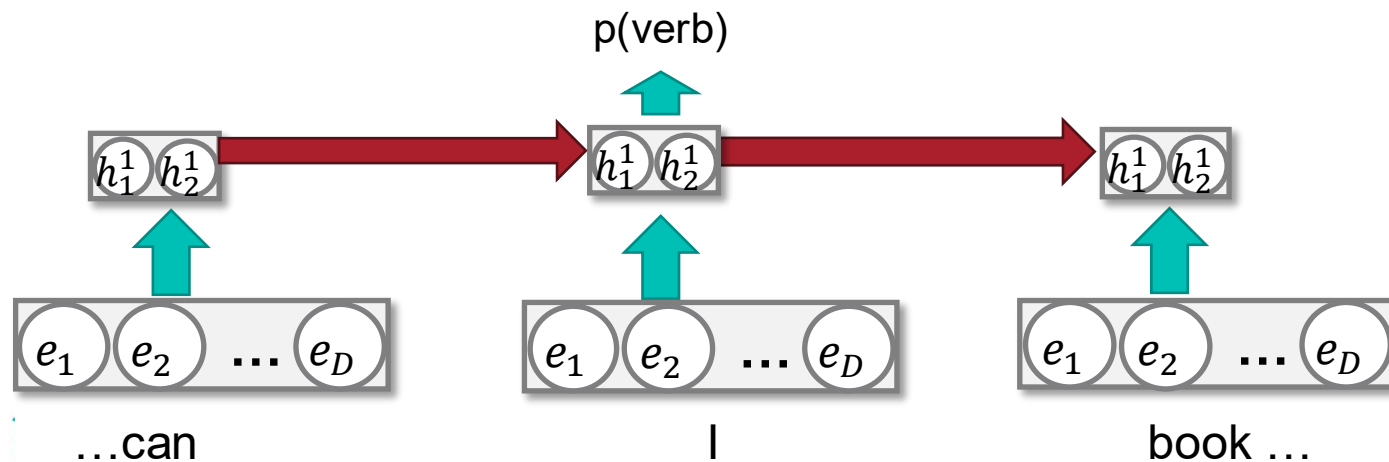


What can you do with these embeddings?

*Token classification,
e.g., identify the verbs*

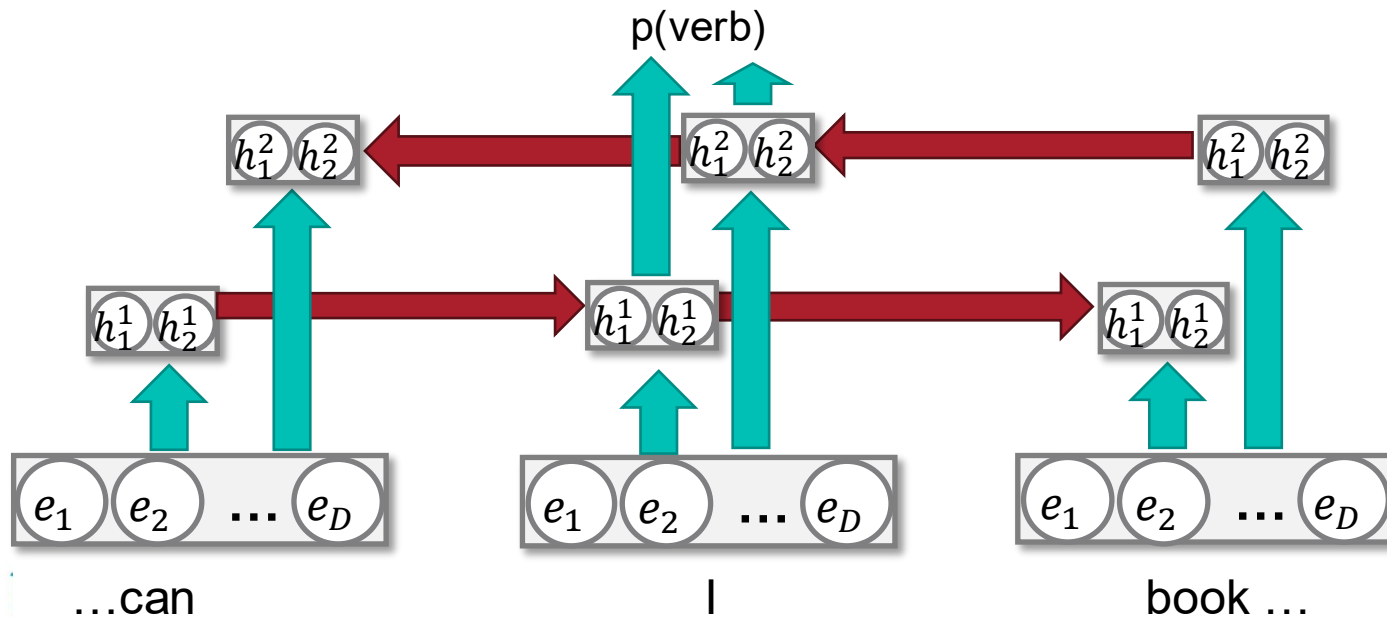


How Can We Improve on Basic RNNs?



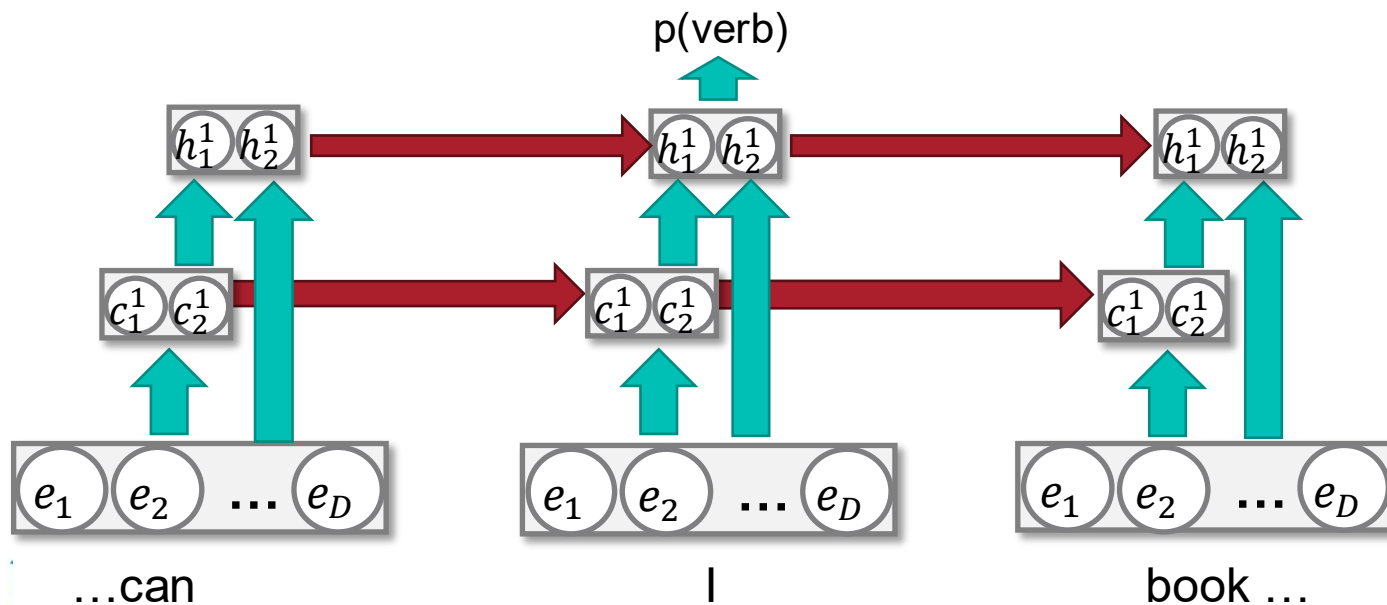
How Can We Improve on Basic RNNs?

Bidirectional: get context from both directions



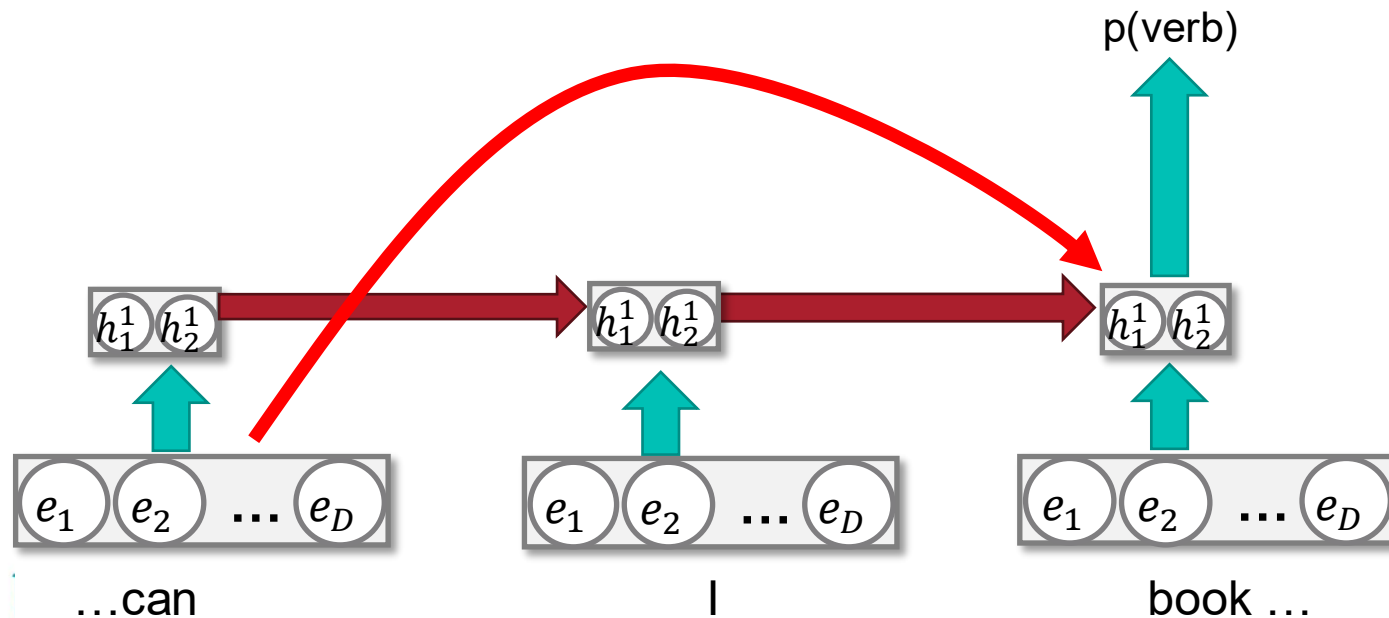
How Can We Improve on Basic RNNs?

LSTM: separate context from word representations



How Can We Improve on Basic RNNs?

Attention: dynamically retrieve information from earlier in the sequence



How Can We Learn Contextualised Embeddings?

- **Self-supervised learning** from massive data, using a new task:
- Could we learn to predict the next word?
- Yes, good if we want to generate...
- ...but this doesn't encode the following context

How Can We Learn Contextualised Embeddings?

- **Masked language modelling:** mask out 15% of the words, then predict the masked words.

The man went to the [MASK] to buy a [MASK] of milk

shop

bottle

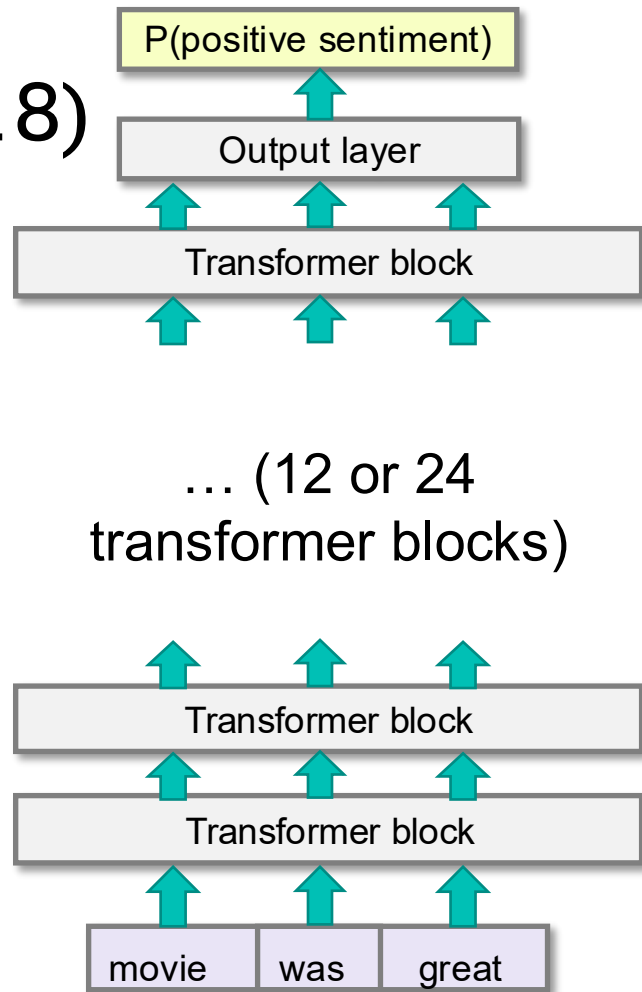
Which words are missing?
What did you need to know
to guess them?

[BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding](#). Devlin et al., 2018.

Google's BERT Model (2018)

- RNNs don't scale well to large dataset as they process tokens sequentially
- Instead, use **transformers** with many stacked hidden layers to **encode** text to contextualised embeddings
- Each block outputs a sequence of contextual token embeddings

[BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.](#) Devlin et al., 2018.

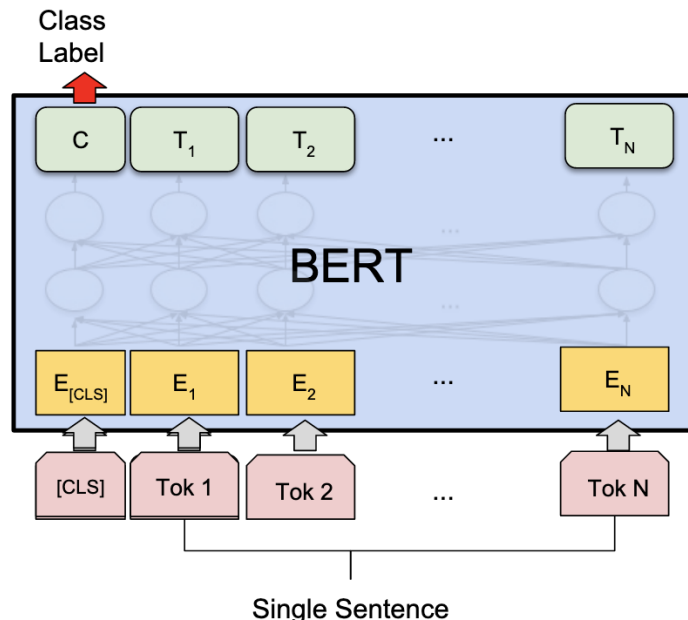
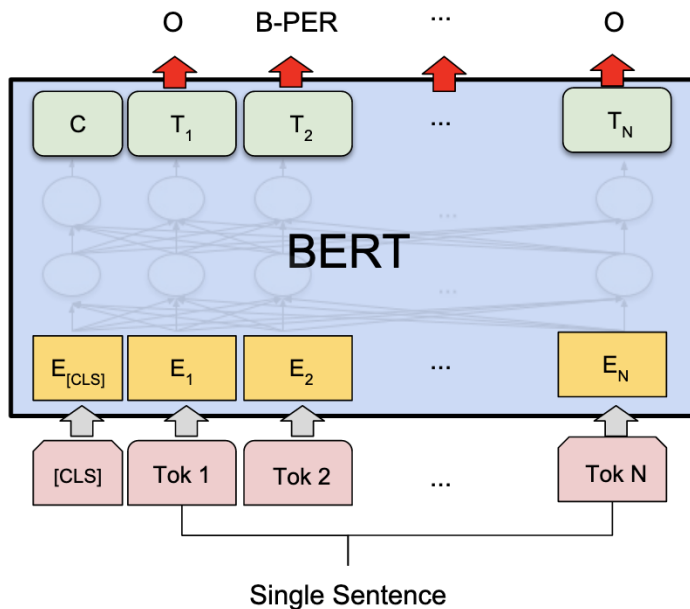


How Do We Use Embedding Models in Practice?



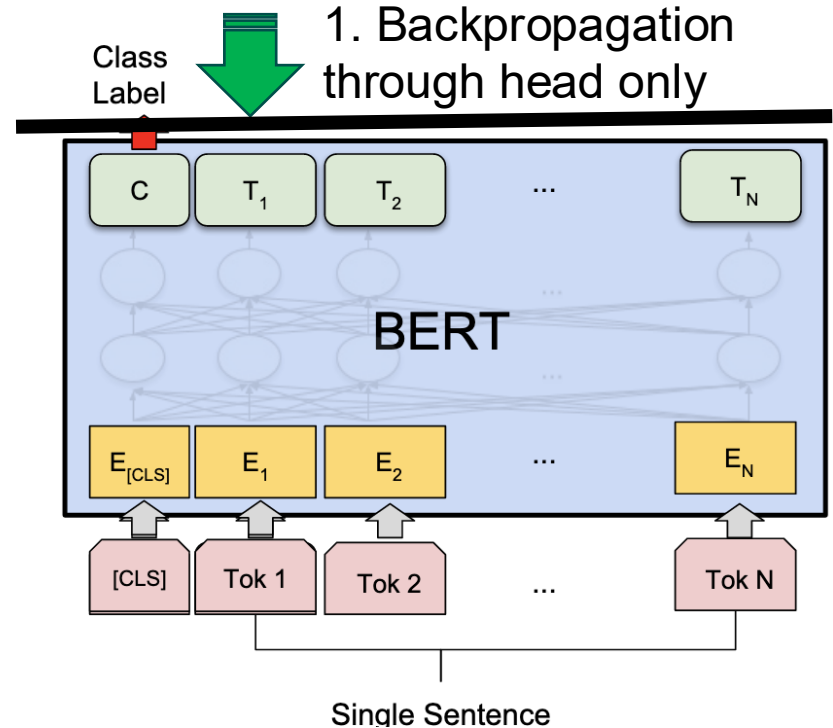
One Base Model, Many Tasks

- Pass embeddings from final layer through task-specific 'head' layer,
e.g., Token classification: Sequence classification:



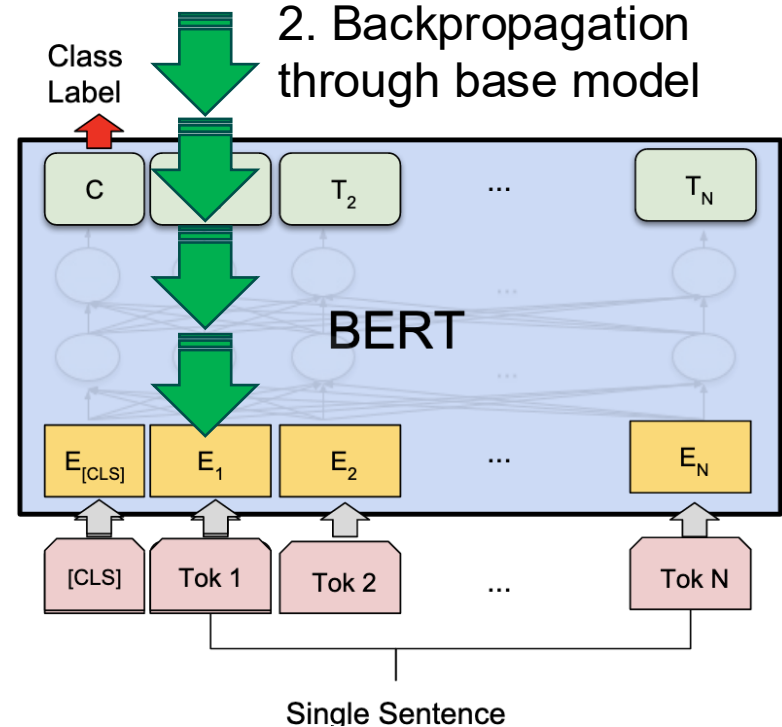
Transfer

- Method 1. **Freeze** the encoder when training downstream model



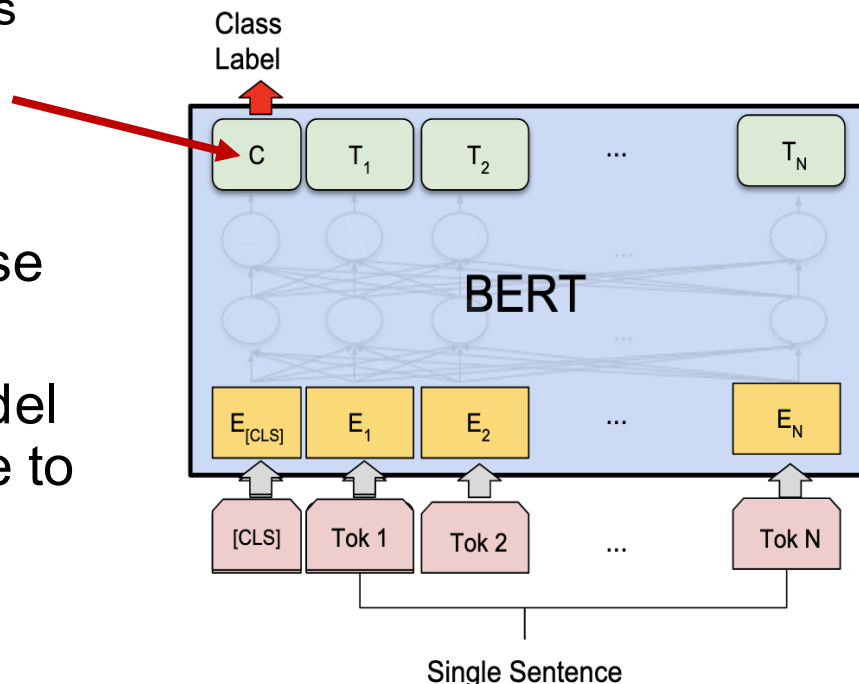
Transfer Learning via Fine-tuning

- Method 1. **Freeze** the encoder when training downstream model
- Method 2. **Fine-tune** the base model to adapt the representations to your target task.
 - Common to tune only top layers
 - Or a small set of ‘adapter’ weights (e.g., LoRA)
- Advantages/disadvantages?
 - Cost/performance trade-off
 - Target task data available?
 - Risk of overfitting/forgetting



How Can We Get a Sentence Embedding?

- Options:
 - Take the special classification token's embedding
 - Average token embeddings
- Both options work with contextual embeddings because they compose meaning across a sequence
- MLM pretraining not optimal – model doesn't learn how sentences relate to one another



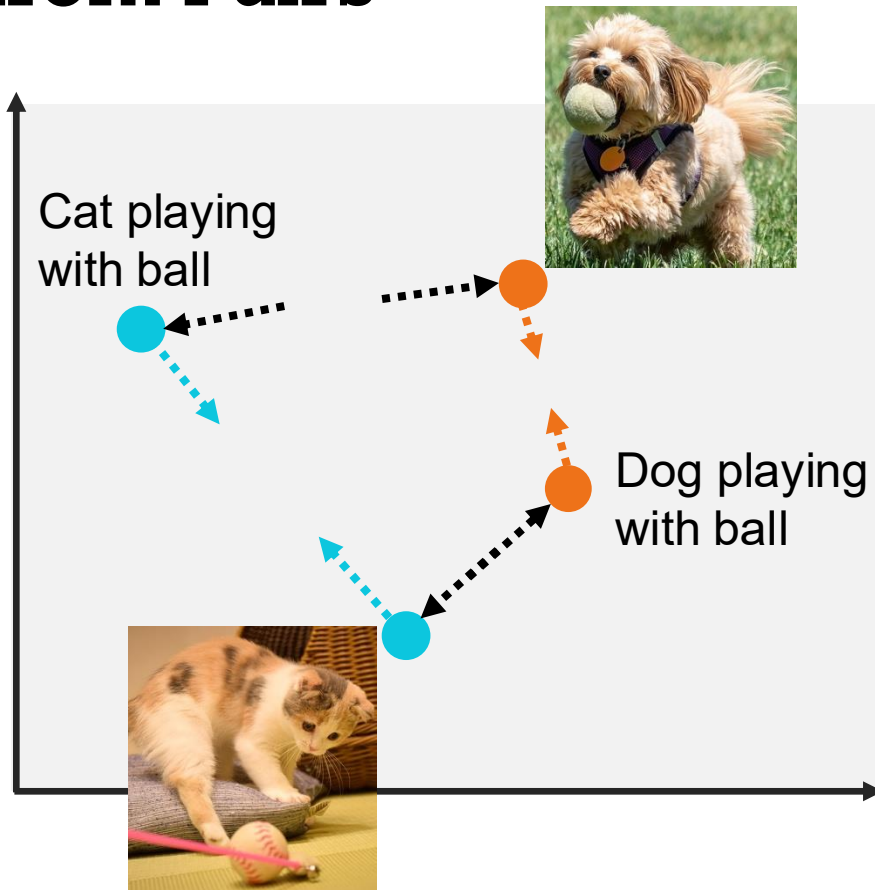
Improving the Embeddings

...in targeted ways



Contrastive Learning from Pairs

- Sentence embeddings from BERT reflect similarity poorly
- SBERT learns from human-labelled sentence pairs using a **contrastive loss** that
 - Pulls similar pairs together
 - Pushes dissimilar pairs apart
- CLIP uses same approach to align image embeddings to matching caption embeddings [Radford et al., ICML 2021]



Limits of Sentence Embeddings: Factuality

- Sentence embedding reflect differences in meaning despite similar words, e.g.:
- I **like** rainy days because they make me feel **relaxed**.
- I **don't like** rainy days because they don't make me feel relaxed.

Token-level BERTScore	0.984
-----------------------	-------

Sentence BERTScore	0.720
--------------------	-------

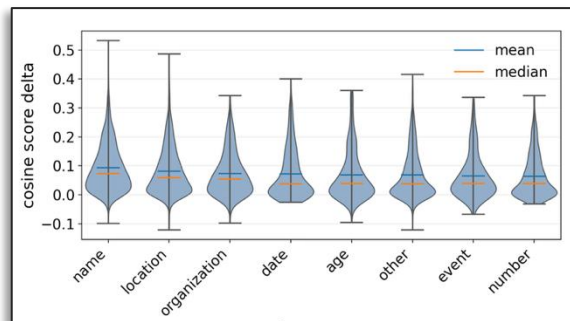
- But similarity $\neq$ factual agreement
- **Source:** The patient was prescribed **10 mg** of drug X.
- **Summary:** The patient was prescribed **20 mg** of drug X.

Factuality gold label:	wrong
------------------------	-------

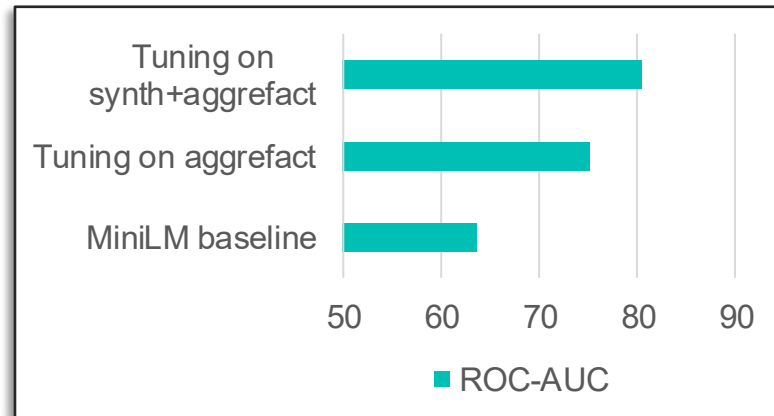
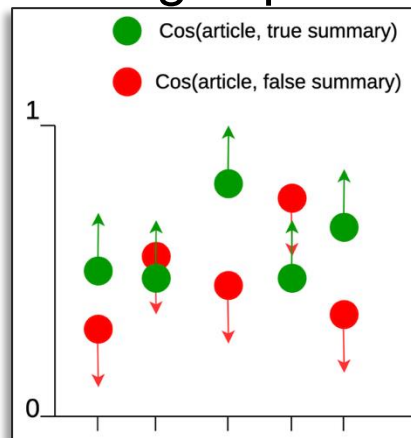
Sentence BERTScore	???
--------------------	-----

Improving Embeddings for Factuality

- Generate factual errors using in summaries by using an LLM to mutate named entities
- Fine-tune sentence embeddings with contrastive learning
- Zero-shot factuality checking improves on AggreFact but needs in-domain tuning



bristol.ac.uk



Open Questions

- Representing and detecting factual differences in embeddings
- How to represent and explain uncertainty in embeddings
 - Probability distributions rather than point values?
 - Confidence signals inside the model
- Interpreting figurative or subjective language
- Transferring knowledge into small, efficient models
- Connecting modalities, background knowledge and grounding into a single model

Summary

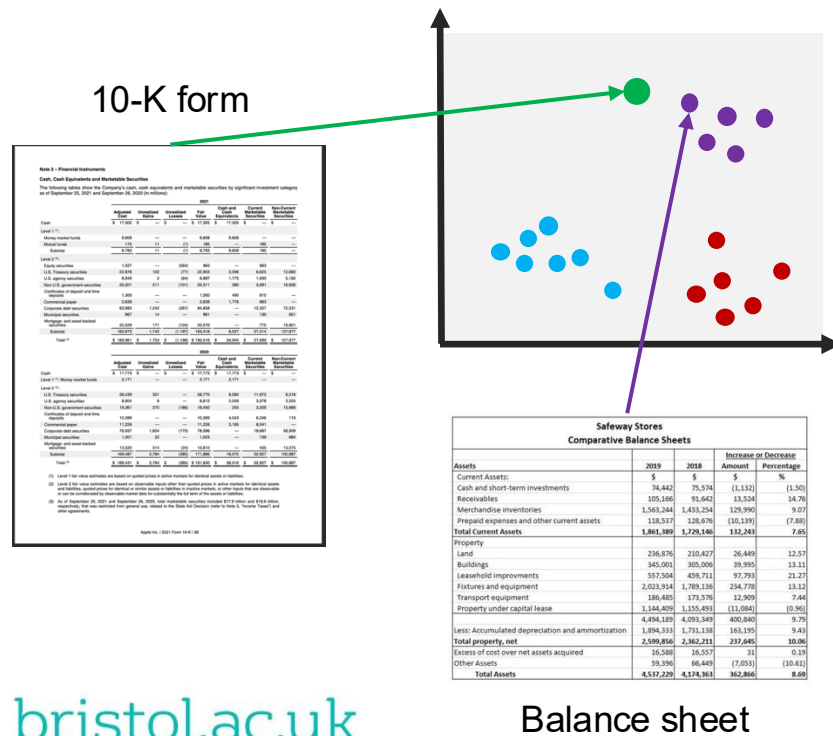
- Embeddings represent meaning as vectors
- Self-supervised learning scales data and models without labelling
- Transfer learning lets us reuse what we have learned for new tasks
- Contrastive learning is a versatile tool for improving embeddings
- Representations are vital for building your AI system: they should be as expressive as you need and as compact as you can make them to enable your model to generalise from training examples.

Out-of-Distribution detection (extra slides)

How does a model handle something at inference time that it hasn't seen before?



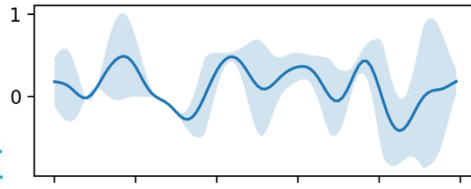
Out-of-Distribution (OOD) Classes



- Model learns an embedding space that separates training examples by class
- New type of document appears at test time, but the classifier wasn't trained for this!
- Matches **over-confidently** and incorrectly to the closest in-distribution class

Why are OOD Inputs Difficult To Recognise?

- Model only learns to **discriminate** between in-distribution (ID) classes
- Model may focus on misleading discriminative features
- Ignores clues that document is different
- Uncertainty of OOD examples is underestimated



Note 3 - Financial Instruments
Cash, Cash Equivalents and Marketable Securities
The following tables show the Company's cash, cash equivalents and marketable securities by significant investment category as of September 25, 2021 and September 26, 2020 (in millions).

	Adjusted Cash	Unadjusted Cash	Unadjusted Cash	Cap and Cash	Current	Non-Current
	\$	\$	\$	\$	\$	\$
2021						
Cash	\$ 17,360	\$	\$ 17,360	\$	\$ 17,360	\$
Level 1 (1)						
Money market funds	8,628	—	8,628	8,628	—	—
Marketable securities	8,732	11	8,743	8,732	11	—
Subtotal	\$ 17,360	\$ 11	\$ 17,371	\$ 17,360	\$ 11	\$
Level 2 (1)						
Equity securities	1,527	—	(544)	983	—	983
U.S. Treasury securities	20,878	162	(77)	20,963	3,396	6,625
U.S. agency securities	8,969	2	446	8,967	1,773	1,330
Non-U.S. government securities	30,291	211	(161)	30,341	395	3,691
Corporate debt securities	1,368	—	—	1,368	458	810
Commercial paper	2,439	—	—	2,439	1,776	663
Corporate debt securities	60,893	1,262	(247)	60,908	—	12,357
Multiple securities	987	14	—	981	—	130
Marketable securities	75,572	1,389	(1,012)	75,949	3,729	19,801
Subtotal	\$ 92,932	\$ 1,400	\$ (1,001)	\$ 93,331	\$ 3,740	\$ 20,731
Total (1)	\$ 109,892	\$ 1,411	\$ (990)	\$ 109,313	\$ 3,780	\$ 20,731
2020						
Cash	\$ 17,312	\$	\$ 17,312	\$	\$ 17,312	\$
Level 1 (1)						
Money market funds	8,715	—	—	8,715	8,715	—
Level 2 (1)						
Equity securities	28,428	331	—	28,759	8,366	11,875
U.S. Treasury securities	8,804	8	—	8,812	2,338	3,078
U.S. agency securities	16,361	275	(186)	16,450	233	3,529
Non-U.S. government securities	12,268	—	—	12,268	3,182	8,947
Corporate debt securities	16,507	1,824	(175)	16,356	—	19,687
Commercial paper	1,581	82	—	1,663	130	886
Multiple securities	13,535	214	(245)	13,504	—	435
Marketable securities	70,796	2,334	(426)	72,604	3,648	26,835
Subtotal	\$ 88,108	\$ 2,334	\$ (426)	\$ 89,916	\$ 3,648	\$ 26,835
Total (1)	\$ 105,420	\$ 2,334	\$ (426)	\$ 107,328	\$ 3,648	\$ 26,835

(1) Level 1 fair value estimates are based on quoted prices for identical assets or liabilities.
(2) Level 2 fair value estimates are based on observable inputs other than quoted prices for identical assets or liabilities. Quoted prices for identical or similar assets or liabilities in active markets, or other inputs that are observable or can be corroborated by observable market data for identical or similar assets or liabilities.
(3) As of September 25, 2021 and September 26, 2020, total marketable securities included \$773 million and \$518 million, respectively, that was restricted from general use, related to the State Aid Restriction (see Note 7 "Restrictions") and other agreements.

Text content of footer = 'Form 10-K'

Table layout

Safeway Stores
Comparative Balance Sheets

Assets	2019	2018	Increase or Decrease
	\$	\$	Amount Percentage
Current Assets:			
Cash and short-term investments	78,442	75,574	(1,132) (1.50)
Receivables	105,166	91,842	13,324 14.76
Merchandise inventories	1,563,244	1,433,254	129,990 9.07
Prepaid expenses and other current assets	118,537	128,076	(10,139) (7.88)
Total Current Assets	1,865,389	1,729,146	132,243 7.65
Property:			
Land	236,876	210,427	26,449 12.57
Buildings	345,001	305,006	39,995 13.11
Leasehold improvements	357,504	439,711	(82,207) (18.92)
Furniture and equipment	2,021,914	1,793,136	228,778 12.76
Transport equipment	186,483	173,576	12,907 7.44
Property under capital lease	1,144,409	1,155,493	(11,084) (0.96)
Total Property	4,494,187	4,093,349	400,838 9.79
Less: Accumulated depreciation and amortization	(2,894,333)	(2,731,130)	(163,203) (5.94)
Total Assets	4,515,259	4,174,361	340,898 8.19

Can We Recognise OOD Inputs Without Retraining?

- Intervening at training time is not always possible
- To avoid modifying the training of a model, OOD detection can be applied at inference time:

OOD Detection Process

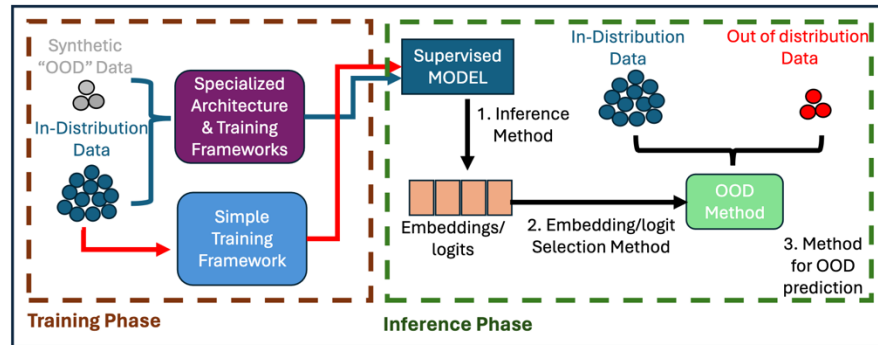
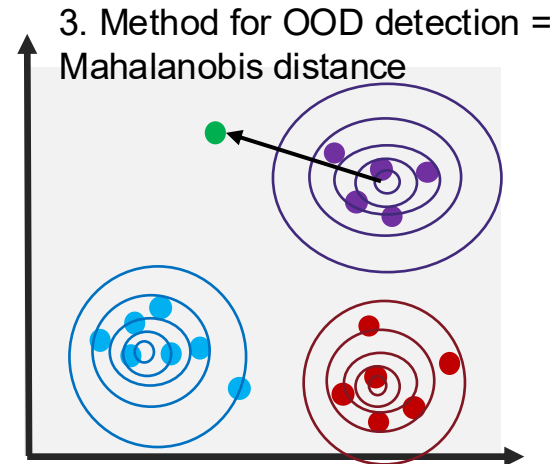


Fig. 3. Schematic of OOD optimization methods in OOD Process. Training and Inference Phases. There are three stages in the Inference Phases of OOD process. Stage one of the process explores different inference methods for generating logits or embeddings from the fine-tuned model. Stage two focuses on how to effectively aggregate the generated logits or embeddings. Stage three is the actual OOD method that creates the decision boundary between ID and OOD data using the outputs of stage two.



Attention Head Masking (AHM)

- Prevent reliance on small number of features by dropping out attention heads
- Run T inference passes
- Each time, mask random attention heads and obtain embeddings
- Select F embeddings most similar to training examples, then average them to obtain embedding for OOD detection

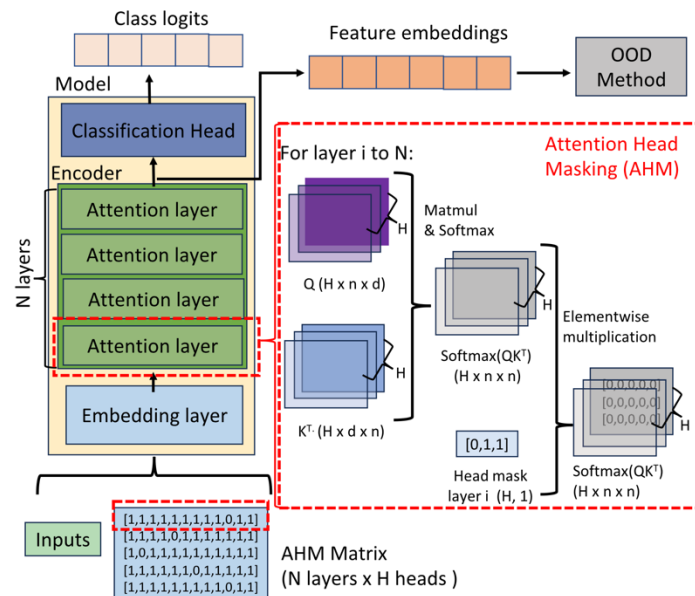


Fig. 1. Visual demonstration of AHM on a transformer-based model: For each attention layer, we utilize the corresponding attention head mask from the AHM matrix. Following query-key multiplication and the subsequent softmax operation, the resulting attention scores undergo element-wise multiplication with the relevant attention head mask. This process effectively reduces the attention scores of certain heads to zero, thereby inhibiting the propagation of their respective information through the value matrix.

AHM Improves OOD Detection Within and Across Datasets

- AHM brings large **~10% drops in false positive rate** (when OOD cases fail to be detected)
- Reductions seen across and within datasets (tobacco3482 document scans, finance docs)
- Uni/multi-modal with layout+image

Method	Tobacco3482 (ADVE OOD)		Tobacco3482 (Cross-dataset OOD)		FinanceDocs (Resume OOD)		FinanceDocs (Cross-dataset OOD)	
	AUROC (%)	FPR (%)	AUROC (%)	FPR (%)	AUROC (%)	FPR (%)	AUROC (%)	FPR (%)
knn _{CLS}	95.8 ± 1.1	26.9 ± 7.4	99.1 ± 0.4	3.0 ± 1.8	96.5 ± 2.3	17.2 ± 12.7	89.1 ± 1.7	58.9 ± 6.7
knn _{CLS} _AHM (Ours)	96.9 ± 0.9	18.2 ± 3.9	99.1 ± 0.3	2.4 ± 1.3	97.5 ± 1.4	11.4 ± 8.8	88.5 ± 1.1	56.2 ± 9.6
Mahalanobis _{CLS}	97.6 ± 0.9	15.5 ± 5.3	99.6 ± 0.2	1.0 ± 0.9	97.7 ± 1.3	12.2 ± 10.0	89.8 ± 1.7	54.1 ± 9.0
Mahalanobis _{CLS} _AHM (Ours)	98.5 ± 0.5	7.1 ± 4.1	99.7 ± 0.2	0.6 ± 0.6	97.8 ± 1.2	9.9 ± 8.6	89.2 ± 1.3	52.2 ± 12.6
Mahalanobis _{AvgAvg}	94.2 ± 0.8	37.5 ± 5.4	99.7 ± 0.1	0.04 ± 0.05	99.6 ± 0.3	0.6 ± 0.5	94.9 ± 1.5	35.3 ± 19.6
Mahalanobis _{AvgAvg} _AHM (Ours)	95.6 ± 0.7	26.7 ± 0.7	99.8 ± 0.1	0.01 ± 0.09	99.6 ± 0.3	0.4 ± 0.3	95.1 ± 1.2	30.2 ± 1.2

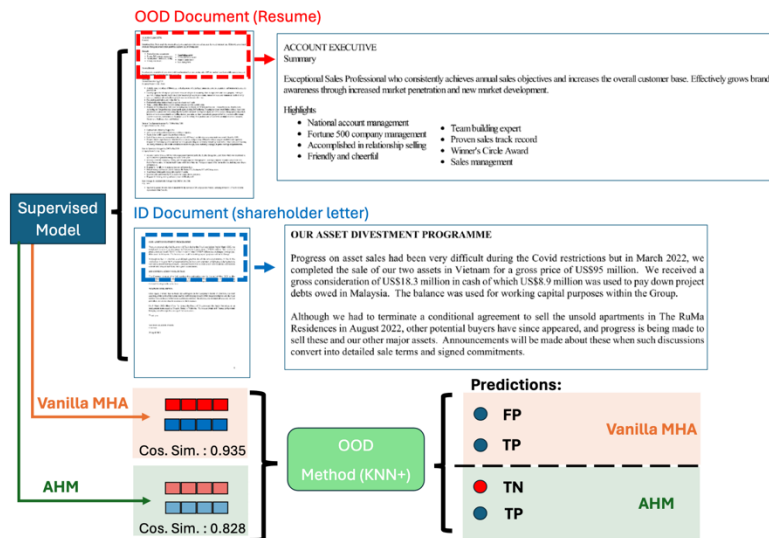


Fig. 2. Illustration of the effectiveness of AHM in reducing false positive rates on real-world samples. The blue-highlighted example represents an in-distribution (ID) class, “shareholder letter”, while the red-highlighted example corresponds to an out-of-distribution (OOD) class, “resume”. This sample pair demonstrates a case where low similarity between the embedding representations of ID and OOD data is expected. The orange path illustrates the embeddings and corresponding predictions obtained using standard Multi-Head Attention (Vanilla MHA), whereas the green path represents the embeddings and predictions generated when Attention Head Masking (AHM) is applied.